

ZK-KYA: A Privacy-Preserving Identity and Access Layer for AI Agents

COMS 4995-001: Science of Blockchain | Project Proposal | Spring 2026

Aaditya Pai, Yegan Dhaivakumar, Jie Wang, Rujing Li

1. Problem Statement and Motivation

AI agents are moving from research prototypes to deployed systems that transact, retrieve sensitive records, and take actions on behalf of users. Consider a user who deploys an agent to pay their electricity bill. The agent needs to access a bank account, identify the correct payee, and execute a transfer within a bounded dollar amount and time window. The bank, on its end, needs some way to verify that this agent is actually authorized to do what it claims, without the user being online at the time of the transaction.

Today there is no good way to do this. Existing approaches like OAuth tokens and API keys depend on centralized intermediaries to enforce permissions. These are vulnerable to compromise, coercion, and operational failure, and they require agents or intermediaries to hold sensitive details in plaintext. As a result, agents today either operate with overly broad access or get blocked entirely by services that cannot verify automated requests. As agentic systems start interacting with finance, healthcare, and enterprise software, this becomes a real bottleneck.

We think this problem maps naturally onto blockchain infrastructure. On-chain credential registries give you tamper-resistance and independent verifiability without a trusted third party. Combined with zero-knowledge proofs, they let an agent prove it holds a valid authorization without revealing the underlying credential details. This project proposes **ZK-KYA (Zero-Knowledge Know Your Agent)**, a proof-of-concept system that combines digital signatures, a smart contract registry, and zk-SNARKs to build a working identity and authorization layer for autonomous agents.

2. Technical Approach

The system has three cryptographic components. Each one ties back to topics from the course.

2.1 Credential Issuance and On-Chain Signature Verification

Users issue permissions by signing a structured credential with ECDSA over secp256k1, the same signature scheme Ethereum uses for transaction authorization (covered in Lecture 2). Each credential encodes the agent's Ethereum address, the set of permitted actions, the target service, a spending cap, and an expiration timestamp. The signature binds the credential to the user's on-chain identity, making it unforgeable. When a credential is submitted, the smart contract verifies the ECDSA signature on-chain using Ethereum's native *ecrecover* precompile, which recovers the signer's address from the signature and confirms the credential was issued by the claimed user. This keeps signature verification simple, gas-efficient, and outside the ZK circuit (more on this separation below).

2.2 On-Chain Credential Registry (Solidity Smart Contract)

Once the signature is verified, the contract stores a Poseidon hash commitment of the credential on-chain rather than the credential itself. This is the key privacy mechanism: the chain records that a valid credential exists, but not what it says. The contract exposes two main functions: *issueCredential()*, which verifies the ECDSA signature and stores the hash commitment, and *verifyProof()*, which accepts a zk-SNARK proof from an agent and checks it against the on-chain verifier. Once deployed, the contract is immutable, so no intermediary can retroactively alter issued permissions. The contract design draws on the Ethereum execution layer and account-based state model from Lectures 8 and 9.

2.3 Zero-Knowledge Authorization (Groth16 via circom/snarkjs)

When an agent wants to perform an action, it needs to prove it holds a valid credential without revealing the credential itself. We handle this with a circom circuit that proves three things: (1) the prover knows a credential whose Poseidon hash matches one of the commitments stored on-chain, (2) the credential has not expired relative to the current block timestamp, and (3) the requested action falls within the credential's permitted scope (correct service, action type within bounds, amount under the spending cap).

An important design decision: we deliberately keep ECDSA signature verification out of the ZK circuit. Verifying ECDSA inside circom requires non-native field arithmetic (secp256k1 operates over a different field than bn128), which would balloon the constraint count into the hundreds of thousands and make proof generation slow. Instead, the contract handles signature verification at issuance time using *ecrecover*, and the ZK circuit only needs to prove knowledge of a credential matching a commitment that the contract has already validated. This separation keeps the circuit small and practical while preserving the security guarantee: if a commitment is on-chain, the contract has already confirmed it came from a legitimate user.

We plan to use Groth16 as the proving system. Groth16 produces constant-size proofs (around 200 bytes) and can be verified efficiently on Ethereum using the bn128 pairing precompile, which makes on-chain verification practical. The trade-off is a per-circuit trusted setup, which is acceptable for a proof-of-concept. If time allows, we will also benchmark PLONK to compare proving times and see what removing the trusted setup costs in practice. This component connects to the SNARKs and validity rollups material from Lecture 20.

3. System Architecture

The proof-of-concept has five components: (1) a **credential registry contract** in Solidity for signature verification, commitment storage, and on-chain proof verification; (2) an **identity module** for ECDSA key management and credential signing, built with ethers.js; (3) a **circom/snarkjs ZK circuit** that proves credential validity, expiration, and scope without revealing the credential; (4) a **mock service API** (Node.js) representing a bank or healthcare provider that queries the on-chain verifier before allowing an agent action; and (5) a **React frontend** where users issue credentials, configure permission scopes, and view an audit log of agent activity.

4. Proposed Demo and Evaluation

We will demonstrate two end-to-end scenarios against the same deployed contract. First, a **financial agent** paying a utility bill: the user issues a scoped credential, the agent generates a ZK proof, the mock bank verifies it on-chain, and the payment goes through. Second, a **healthcare agent** retrieving a patient record, same proof flow but with a different permission scope. In both cases we will also show a rejected action (agent tries to exceed its permissions) and automatic expiration (credential becomes invalid after its timestamp passes). For evaluation, we will measure proof generation time, proof size, on-chain verification gas cost, and total RICS constraints, and use these to discuss what would be needed for production-scale deployment.

5. Connection to Course Material

This is an applied project, but it draws on several foundational topics from the course. The ECDSA credential layer connects to the digital signatures material from Lecture 2. The smart contract design builds on the Ethereum execution layer from Lectures 8 and 9, including how transactions interact with on-chain state and how the EVM processes verification logic. The Groth16 integration ties into the SNARKs content from Lecture 20, including pairing-based verification and RICS constraints. The privacy design (hash commitments on-chain, credentials kept private) is closely related to the approach taken by Zcash and Aleo (suggested project topics 12 and 13). The project also touches on transaction fee considerations from Lecture 6, since on-chain proof verification has a direct gas cost we need to measure and manage.

6. Why This Matters

The gap between what AI agents can technically do and what they are authorized to do is only going to get wider. ZK-KYA is a proof-of-concept, but it shows a model where permissions are enforced through cryptographic rules rather than centralized policy servers. The key property is that once a credential is issued on-chain, its enforcement does not depend on any single operator staying honest or online. We think this is a meaningful direction for blockchain infrastructure and a place where the tools from this course, signatures, smart contracts, SNARKs, and on-chain verification, come together in a practical way.